

COMP3308/3608 Week 13 - Recommender Systems

Author

K. Yu

2026

Contents

1	Introduction & Motivation	2
2	Collaborative Filtering (CF)	2
2.1	Family of CF methods	3
2.2	User-to-User (neighbourhood) CF	3
2.3	Item-to-Item CF	4
2.4	Can we precompute the similarities?	5
2.5	Explicit vs. implicit ratings	5
3	Advanced CF Methods	5
3.1	SVD-based CF (dimensionality reduction)	5
3.2	Matrix Factorisation	6
3.3	Probabilistic CF (Naive Bayes)	7
3.4	Clustering-based CF	7
3.5	Association-rule-based CF	7
3.6	CF — Strengths & Weaknesses	8
4	Content-Based (CB) Recommenders	8
4.1	As a classification/regression task	8
4.2	Representing text items — tf-idf	9
4.3	Practical improvements	9
4.4	Classifiers used	10
4.5	CB — Strengths & Weaknesses	10
5	Challenges in RecSys	10
6	Modern Industry Methods (Not Examinable)	10
6.1	Neural Collaborative Filtering (NCF)	11
6.2	Two-Tower (duel-encoder) model	11
6.3	Wide & Deep	11
6.4	Deep & Cross Network (DCN / DCN-v2)	12

1 Introduction & Motivation

Recommender systems (RecSys) recommends *items* to *people* and predicts their relevance (which camera, which movie, which book, which degree, etc.). The output is either a predicted rating for an unseen item, or a ranked **top- N list**.

Why we need it: the web is the primary information source and is enormous $\rightarrow$ **information overload**. RecSys reduce this by predicting relevance, surfacing a short, personalised list instead of everything.

The three main approaches

Approach	Slogan	Uses item features?	Uses community ratings?
Collaborative filtering	"Books popular among my peers"	No	Yes
Content-based	"More of the kind I already like"	Yes	No
Hybrid	combine both	Yes	Yes

Table 1: Recommendation approaches.

The contrast is the conceptual backbone: CF exploits *who agrees with whom*; CB exploits *what an item is*. Their weaknesses are almost complementary, which is why hybrids dominate in practice.

2 Collaborative Filtering (CF)

Core idea — "**wisdom of the crowd**". Predict $r_{a,i}$ for a target user a and unseen item i from the pattern of ratings across the whole matrix R .

- **Input:** the user-item rating matrix R (assumes a has rated *some* items).
- **Output:** a predicted rating $\hat{r}_{a,i}$, and/or top- N list sorted by score.



Figure 1: Alice's *actual* ratings (top) and the system's *predicted* ratings for unseen movies (bottom).

2.1 Family of CF methods

- **Nearest-neighbour (memory-based):** user-to-user, item-to-item.
- **Model-based/advanced:** matrix factorisation (SVD-style), probabilistic (Naive Bayes), clustering-based, association-rule-based.

2.2 User-to-User (neighbourhood) CF

Idea. Find a neighbourhood N of users who:

- (a) agreed with a in the past and
- (b) have rated item i ;

combine their ratings on i .

Three design questions the method must answer:

1. How to measure user-user similarity?
2. How many neighbours to use?
3. How to combine neighbour ratings into a prediction?

Worked example (running matrix)

	Item1	Item2	Item3	Item4	Item5
Alice	5	3	4	4	?
User1	3	1	2	3	3
User2	4	3	4	3	5
User3	3	3	1	5	4
User4	1	5	5	2	1

Goal: predict whether Alice will like item 5.

1. Similarity — Pearson correlation

Most common in *user-based* CF. For users a, b with set of co-rated items P :

$$\text{sim}(a, b) = \frac{\sum_{p \in P} (r_{a,p} - \bar{r}_a)(r_{b,p} - \bar{r}_b)}{\sqrt{\sum_{p \in P} (r_{a,p} - \bar{r}_a)^2} \sqrt{\sum_{p \in P} (r_{b,p} - \bar{r}_b)^2}}.$$

Pearson is preferred over raw cosine because **subtracting each user's mean removes rating bias** (a "tough" rater who gives 2-3 vs. a "generous" rater who gives 4-5 can still be perfectly correlated).

Computing against Alice (co-rated items = Item1-4, $\bar{r}_{\text{Alice}} = 4$):

Neighbour	sim(Alice, ·)
User1	0.85
User2	0.70
User3	0.00
User4	-0.79

(e.g. for User1: deviations $[1, -1, 0, 0]$ vs $[0.75, -1.25, -0.25, 0.75]$ give numerator 2.0 and $\text{sim} = 2.0/(\sqrt{2} \cdot \sqrt{2.75}) \approx 0.85$)

2. How many neighbours?

Either a **similarity threshold** or a **fixed k** (here $k = 2$). By the values above the two nearest neighbours are **User1 (0.85)** and **User2(0.70)**.

3. Combining ratings

- **Simple average** of neighbours' ratings on item5: $(3 + 5)/2 = 4.0$.
- **Weighted, mean-centred (adjusted) average**—the *commonly used* formula:

$$\hat{r}_{a,p} = \bar{r}_a + \frac{\sum_{b \in N} \text{sim}(a, b)(r_{b,p} - \bar{r}_b)}{\sum_{b \in N} |\text{sim}(a, b)|}$$

Three ideas are baked in: (i) start from a 's own mean $\bar{r}_a$; (ii) each neighbour contributes its **deviation** $r_{b,p} - \bar{r}_b$ (is p above or below *their* norm?), not the raw rating; (iii) deviations are **weighted by similarity** and **normalised** by total similarity.

Plugging in ($\bar{r}_{\text{Alice}} = 4$, $\bar{r}_{U1} = 2.4$, $\bar{r}_{U2} = 3.8$):

$$\hat{r}_{\text{Alice},5} = 4 + \frac{0.85(3 - 2.4) + 0.70(5 - 3.8)}{0.85 + 0.70} = 4 + \frac{0.51 + 0.84}{1.55} \approx 4.87.$$

So Alice is predicted to *like* Item5 strongly.

Refinements (why naive neighbourhoods underperform)

- **Variance / inverse-user-frequency weighting:** agreement on a *universally liked* item is uninformative; agreement on a *controversial* (high-variance) item is strong evidence of like-mindedness $\rightarrow$ up-weight high-variance items.
- **Co-rated count:** if two users share very few co-rated items, their similarity is statistically unreliable $\rightarrow$ shrink/down-weight it.
- **Case amplification:** raise similarities to a power so that **very** similar neighbours ($\text{sim} \approx 1$) dominates, suppressing weak neighbours.

2.3 Item-to-Item CF

Idea. Instead of similar users, find items similar to the target item, then predict from the target user's own ratings on those similar items.

Similarity — (adjusted) cosine

Most common in *item-based* CF. Ratings on items a, b are vectors; cosine = angle between them:

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a}^\top \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}.$$

Plain cosine ignores that different users use different rating scales. **Adjusted cosine** subtracts each *user's* mean $\bar{r}_u$ before comparing items (over U = users who rated both items):

$$\text{sim}(a, b) = \frac{\sum_{u \in U} (r_{u,a} - \bar{r}_u)(r_{u,b} - \bar{r}_u)}{\sqrt{\sum_{u \in U} (r_{u,a} - \bar{r}_u)^2} \sqrt{\sum_{u \in U} (r_{u,b} - \bar{r}_u)^2}}.$$

Prediction — weighted sum over rated items

$$\hat{r}_{u,p} = \frac{\sum_{i \in \text{ratedItems}(u)} \text{sim}(i, p) r_{u,i}}{\sum_{i \in \text{ratedItems}(u)} \text{sim}(i, p)},$$

i.e. blend the user's existing ratings, weighting each by how similar that item is to the target p .

2.4 Can we precompute the similarities?

The matrix has **many items, few ratings per user**. This makes user-side vs item-side similarities behave very differently:

- **User-to-user:** similarity is **unstable** — adding a few ratings changes the small co-rated set and shifts sim a lot. $\implies$ precomputing user neighbourhoods gives poor results; must compute online.
- **Item-to-item:** similarity is **stable** (items accumulate many ratings) $\implies$ can **precompute** the item-item similarity matrix and neighbours offline; serving = lookup + weighted sum. **This is essentially what Amazon does** (and is the scalability reason item-based CF won industrially).

2.5 Explicit vs. implicit ratings

- **Explicit:** the user states a rating (typically 1-5 or 1-7 Likert). Reliable, but **hard to collect enough of** $\implies$ sparse matrix $\implies$ poor quality. Big design question: *how to encourage rating?*
- **Implicit:** inferred automatically from behaviour — purchases, clicks, page views, dwell time, demo downloads. A purchase is often read as a positive signal. **Risk:** behaviour is ambiguous (bought a gift for someone else; bought but disliked). Usually **combined** with explicit signals.

3 Advanced CF Methods

3.1 SVD-based CF (dimensionality reduction)

Motivation. Reduce the dimensionality of $R \implies$ faster predictions, denser latent representation. Singular Value Decomposition is the same tool used for feature selection, image compression, and

Latent Semantic Analysis in 1R.

SVD. Any $n \times m$ matrix X with $n \times m$ factorises as

$$X = U\Sigma V^\top$$

- U ($n \times m$, orthogonal) — the **transformed data** (rows = data points in the new axes).
- V ($m \times m$, orthogonal) — defines **new orthogonal axes** ordered by variance (axis 1 = direction of highest variance, axis 2 $\perp$ axis 1 = next,...).
- Σ ($m \times m$, diagonal) — the **singular values** on the diagonal (magnitude of each axis).

Truncation (rank- k). Keep only the k largest singular values $\rightarrow$ keep first k columns of U and first k rows of $V^\top$. This gives the best rank- k approximation $M_k = U_k \Sigma_k V_k^\top$ and yields a small number of **latent features (factors)**.

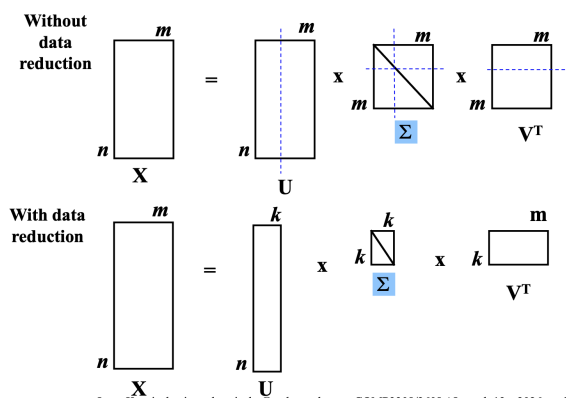


Figure 2: Without data reduction vs with data reduction.

Compression ratio (storage after vs before):

$$r = \frac{nk + k + km}{nm},$$

and for $n \gg m > k$, $r \approx k/m$. We store: first k columns of U , the k singular values, first k columns of $V \Rightarrow k(n + 1 + m)$ numbers.

Prediction. With $k = 2$ latent factors, user/item are points in factor space:

$$\hat{r}_{u,i} = \bar{r}_u + U_k(u) \Sigma_k V_k^\top(i).$$

3.2 Matrix Factorisation

An **extension** of SVD-based CF that fixes SVD's fatal flaw: **classical SVD needs a complete matrix**, but R is mostly missing. Early hacks filled in missing entries before SVD $\rightarrow$ **distortion**

and inaccuracy.

Modern formulation. Learn user factors $\mathbf{p}_u \in \mathbb{R}^k$ and item factors $\mathbf{q}_i \in \mathbb{R}^k$ using **only observed ratings**, by minimising a regularised squared error:

$$(\mathbf{p}^*, \mathbf{q}^*) = \underset{\mathbf{p}, \mathbf{q}}{\operatorname{argmin}} \sum_{(u,i) \in \mathcal{K}} \left[(r_{u,i} - \mathbf{p}_u^\top \mathbf{q}_i)^2 + \lambda (\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2) \right]$$

solved with **stochastic gradient descent** (or ALS). The regulariser λ controls overfitting on the sparse data.

3.3 Probabilistic CF (Naive Bayes)

Treat the unknown rating as a class to predict. For Alice with previous ratings as evidence E and hypotheses $H_v : \text{item5} = v$ for $v \in \{1, 2, 3, 4, 5\}$, apply Bayes with the **conditional independence** assumptions:

$$P(H | E) = \frac{P(H) \prod_{i=1}^d P(E_i | H)}{P(E)}.$$

Estimate each $P(E_i | H)$ from the *other* users. Example ($E = \{\text{I1} = 1, \text{I2} = 3, \text{I3} = 3, \text{I4} = 2\}$), users with Item5=1 are User2 and User4:

$$P(\text{item5} = 1 | E) \propto \underbrace{\frac{2}{2}}_{P(\text{I1}=1|\text{I5}=1)} \cdot \underbrace{\frac{1}{2}}_{\text{I2}=3} \cdot \underbrace{\frac{1}{2}}_{\text{I3}=3} \cdot \underbrace{\frac{1}{2}}_{\text{I4}=2} \approx 0.125.$$

A hypothesis with a *zero* conditional probability (e.g. $\text{item5} = 2$) collapses the whole product to 0 — the usual Naive Bayes zero-frequency issue (fix with smoothing). **Choose the rating v with the highest posterior.**

3.4 Clustering-based CF

Cluster users by their rating vectors (any distance + clustering algorithm). Each cluster is summarised by its **centroid** (component i = average rating of cluster members on item i). For a new user, find the **nearest centroid** and recommend from that cluster's preferences. Trades accuracy for **speed/scalability** (offline clustering, fast online lookup).

3.5 Association-rule-based CF

Borrow market-basket analysis: rules like $X, Y \rightarrow Z$ ("buyers of X, Y also buy Z "). Quality measures over n transactions:

$$\text{support}(X \rightarrow Y) = \frac{\#(X \cup Y)}{n}, \quad \text{confidence}(X \rightarrow Y) = \frac{\#(X \cup Y)}{\#X}$$

Alice example.

1. **binarise** ratings (e.g. 1 if above the user's average, else 0);
2. **mine** rules such as $\text{Item1} \rightarrow \text{Item5}$;

3. keep rules whose LHS matches Alice's transactions;
4. rank candidate items (not yet bought) by **confidence**. *Example:* Item1 $\rightarrow$ Item5 has support 2/4 and confidence 2/2 (excluding Alice).

3.6 CF — Strengths & Weaknesses

Strengths

- Minimal knowledge engineering — needs only R , no item features.
- Captures taste correlations features cannot express.

Weaknesses

- Needs a **user community**.
- Needs enough **co-rated** items; **sparse** matrix hurts.
- **Cold start**: new users/items have no ratings.
- Hard to **explain** recommendations.

4 Content-Based (CB) Recommenders

Idea. Use information about the **items** (not the community): "Recommend fantasy novels to someone who liked fantasy novels." Rooted in **Information Retrieval**. Needs:

1. item content features (genre, actors, director, awards, etc.),
2. a **user profile** (explicit rankings or implicit signals).

Task: recommend items matching the profile.

4.1 As a classification/regression task

For **each user**, treat their rated items as a training set $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$ where $\mathbf{x}^{(i)} \in \mathbb{R}^d$ encodes item features and $y^{(i)}$ is the like/dislike (or rating) label. Train a per-user classifier that predicts the label of new items.

Item's features					class: like/dislike	
Title	Genre	Country	Leading actors	Awards	Keywords or Movie review	Rating
The lives of others	Drama, thriller	German	Sebastian Koch, Martina Gedeck, Ulrich Mühe	Oscar and 66 other awards	In 1984 East Berlin, an agent of the secret police, conducting surveillance on a writer and his lover, ...	5
The Spanish apartment	Comedy, drama, romance	France, Spain	Romain Duris, Jidith Godreche, ...	8 awards	A French student moves into an apartment in Barcelona with six other European students. Together, they speak the international language of love and friendship. ...	5
The sea inside	Drama, biography	Spain	Javier Bardem, Belén Rueda, Lola Dueñas	Oscar and 61 other awards	The real-life story of Ramon Sampedro, who ...	5
Austin Powers	Comedy	USA	Mike Myers, Elizabeth Hurley	3 awards	A 1960s hipster secret agent is brought out of cryofreeze ...	1
...						

Figure 3: The movie table, illustrating item features $\rightarrow$ class label.

4.2 Representing text items — tf-idf

CB is most natural for **text** products (web pages, news), so it often reduces to **text classification**. Standard representation: each document = a vector over selected words, each word weighted by **tf-idf**.

- **tf (term frequency)**: how often a term appears in a document — frequent terms are assumed important. A document becomes a count vector in $\mathbb{N}^{|V|}$, where V is the vocabulary.
- **idf (inverse document frequency)**: down-weights terms common to *all* documents (which carry little discriminative power):

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \cdot \underbrace{\log \frac{N}{\text{df}(t)}}_{\text{idf}}$$

- Can use **bigrams** / **n-grams** instead of single words.

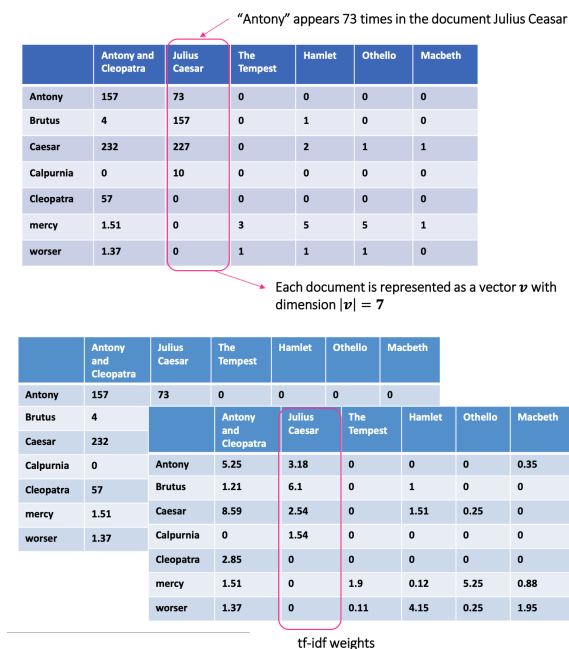


Figure 4: Two Shakespeare-play matrices: raw tf counts and the corresponding tf-idf weights.

4.3 Practical improvements

- **Stop-word removal** — drop “a/the/on/...” that appear everywhere.
- **Stemming** — collapse word variants to a stem (*went* → *go*, *playing* → *play*).
- **Feature selection** — keep only the top- n most representative words (e.g. top 100), since vectors are long and sparse.

4.4 Classifiers used

Most popular: **Naive Bayes**, **SVM**. Others: decision trees, k -NN, linear regression, logistic regression, rule-based, etc.

4.5 CB — Strengths & Weaknesses

Strengths

- No user community required.
- **No cold start** if a profile exists (user stated preferences).
- Often **easier to explain** (depends on classifier).

Weaknesses

- Feature extraction hard in some domains (e.g. music) — CF better there.
- **Over-specialisation**: only items similar to past ones (no **serendipity**).
- Pure CB rarely used alone commercially.

5 Challenges in RecSys

- **Scalability** to real-world data sizes.
- **Short-term vs long-term** preferences.
- **Diversity** of recommendations (counter over-specialisation / filter bubble).
- **Sequence** recommendation (e.g. a playlist), not just isolated items.
- **Mobile** and **group** recommendations.
- **Context-aware** recommendations: travel (winter vs summer), restaurants (Sat vs Mon), in-car music (alone vs with children).
- **Reciprocal (two-way) recommenders** — both sides must like each other (jobs, dating).

6 Modern Industry Methods (Not Examinable)

These deep-learning architectures generalise the classical ideas above and power large-scale production systems. They mostly extend **matrix factorisation** by replacing the simple inner product with learned neural functions, and by ingesting rich side-features $\mathbf{x}$.

6.1 Neural Collaborative Filtering (NCF)

Replaces the MF inner product $\mathbf{p}_u^\top \mathbf{q}_i$ with a learned interaction. Concatenate user/item embeddings and pass through an MLP:

$$\hat{r}_{u,i} = \sigma(\text{MLP}([\mathbf{p}_u; \mathbf{q}_i])).$$

MF becomes a special case.

6.2 Two-Tower (duel-encoder) model

The dominant architecture for **candidate retrieval** at scale (YouTube, large e-commerce). A **user tower** and an **item tower** independently map raw features to embeddings:

$$\mathbf{u} = f_\theta(\mathbf{x}_u) \in \mathbb{R}^k, \quad \mathbf{v} = g_\phi(\mathbf{x}_i) \in \mathbb{R}^k, \quad \text{score} = \mathbf{u}^\top \mathbf{v}.$$

Trained with **sampled-softmax** / **contrastive (in-batch negatives)** loss. The decisive property: **item embeddings $\mathbf{v}$ can be precomputed offline**, and serving becomes **approximate nearest-neighbour (ANN)** search over $\mathbf{u}$ — this is the deep-learning analogue of 2.4 (“precompute item similarities $\rightarrow$ fast lookup”). Handles cold start better than pure CF because the towers consume content features.

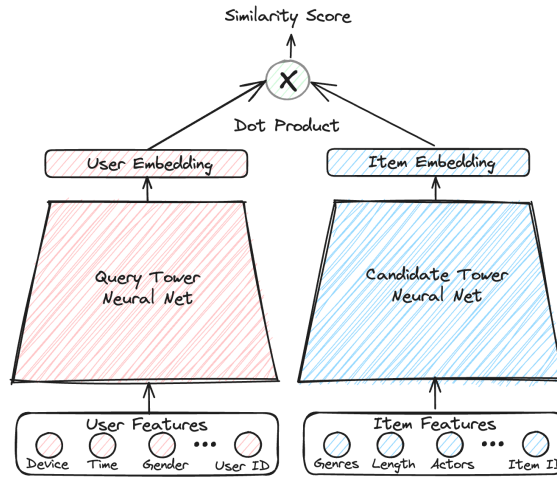


Figure 5: Two-tower architecture.

6.3 Wide & Deep

Jointly trains a **wide** linear model on crossed categorical features (**memorisation** of frequent co-occurrences) and a **deep** MLP on embeddings (**generalisation** to unseen combinations). (*Cheng et al., Google, 2016.*)

6.4 Deep & Cross Network (DCN / DCN-v2)

Automates feature crossing so engineers do not hand-craft interactions. A stack of **cross layers** builds explicit, bounded-degree feature interactions:

$$\mathbf{x}_{l+1} = \mathbf{x}_0 \mathbf{x}_l^\top \mathbf{w}_l + \mathbf{b}_l + \mathbf{x}_l$$

run in parallel with a deep MLP, then combined. Each extra cross layer raises the interaction degree by one, while the residual term $+\mathbf{x}_l$ keeps training stable. (*Wang et al., Google, 2017; DCN-v2, 2020.*)

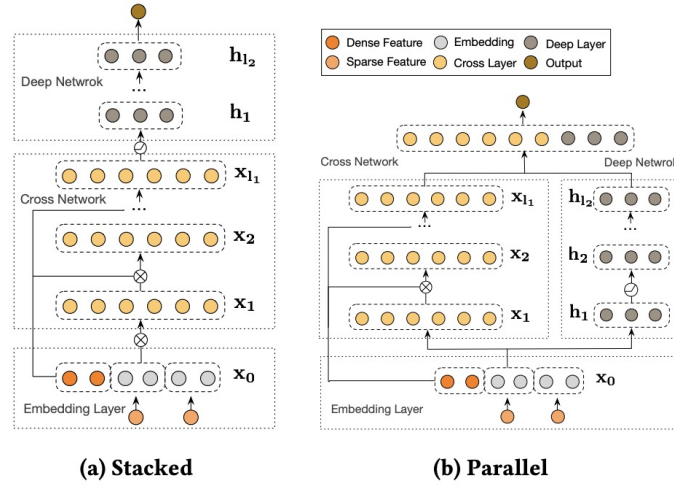


Figure 6: DCN-v2 employs a novel architecture that combines a cross network with a deep network. This combination is realised through two architectures: a stacked structure where the cross network output feeds into the deep network, and a parallel structure where outputs from both networks are concatenated.

Mental map of the lineage: MF ($\mathbf{p}_u^\top \mathbf{q}_i$) $\rightarrow$ NCF (MLP interaction) $\rightarrow$ Two-Tower (separable encoders for retrieval) $\rightarrow$ Wide&Deep / DCN (explicit + implicit feature crossing for ranking). Modern stacks split into **retrieval** (Two-Tower + ANN) $\rightarrow$ **ranking** (Wide&Deep / DCN).